AI6125-MAS GROUP 7 PROJECT REPORT *

Deng Zhouheng
G2504130B
zhuoheng001@e.ntu.edu.sg

Lou Enge
<G2504803B>
enge001@e.ntu.edu.sg

Sun Qingmo
G2503968G
qingmo001@e.ntu.edu.sg

Wu Shanni
<G2502649K>
swu029@e.ntu.edu.sg

Xiong Maihe
<Matric Number>
maihe001@e.ntu.edu.sg

Zhang Yanhai
G2509626L
zh0010ai@e.ntu.edu.sg

ABSTRACT

This report presents Group 7's six-agent system for Tileworld, a partially observable and dynamic grid environment with fuel, sensing, and carrying constraints. Our implementation combines a shared engineering layer (`Group7AgentBase`), protocol-driven collaboration (G7P2), and agent-specific planning policies. We evaluate the system on three parameter sets and report mean reward over repeated runs. Results and ablations show that both communication and policy specialization are necessary for strong team performance, especially in larger and more dynamic maps.

1 Introduction & Background

Tileworld is a standard benchmark for studying multi-agent decision making under uncertainty [1, 2]. Agents operate on a grid with partial observability, stochastic object dynamics, limited fuel, and finite carrying capacity; hence effective behavior requires both local intelligence and team coordination.

In this project, six agents jointly act in the same environment using a Sense→Communicate→Plan→Act loop. We focus on three design questions: (i) how to share local observations efficiently, (ii) how to reduce conflict and duplicated effort, and (iii) how much performance comes from shared infrastructure versus agent-specific policies. The rest of this report presents our implementation, experimental settings, quantitative results, and ablation findings.

2 Environment & Experiment Setup

Our experiment is conducted in a tileworld simulator implemented in Java 1.8 with the Java3D v1.5.1 and MASON 14 package. One simulation will run for 5000 steps and we take the final reward as the metric of the agents system; different simulations will start with a randomly generated seed. The reward is defined by the number of holes filled. The initial fuel is fixed at the maximum fuel carriage of 500.

There are 3 different configurations that affect world size, object lifetime, and object creation distribution.

Table 1: Experimental Parameter Settings

Parameter	World Size	Object Creation Rate	Lifetime
Set 1	50×50	$\mathcal{N}(\mu = 0.2,\ \sigma = 0.05)$	100
Set 2	80×80	$\mathcal{N}(\mu = 2,\ \sigma = 0.5)$	30
Set 3	120×120	$\mathcal{N}(\mu = 5,\ \sigma = 0.5)$	50

* *Contributions*: **Authors are listed alphabetically, all authors contributed to this project equally.**

3 Agent Design

3.1 Group7AgentBase - Abstract Base Class

Group7AgentBase is the shared control and coordination layer for all Group 7 agents. It standardizes low-level navigation (A*-based movement, path reuse, obstacle-aware stepping, and zig-zag fallback exploration), so each agent can focus on high-level policy. It also unifies fuel-safe behavior through common refuel handling. On collaboration, the base implements G7P2 messaging, including directed and broadcast communication, and regularly publishes core shared signals such as discoveries and local sensor snapshots. This design isolates reusable infrastructure from agent-specific decision logic and provides a consistent interface for comparing policies.

3.2 AgentHanny - The Proactive Learner

AgentHanny updates a shared world estimate from local and team observations, then plans by maximizing expected utility.

Planning Actions are selected by **utility over plan**, where each plan is a sequence of pickup/fill decisions from the current state. The key signal is expected utility, computed using survival-based estimates of whether candidate objects remain available at arrival time (derivation in the appendix [3]). The agent executes the highest-EU plan and locks only the currently executed target to reduce conflict. The overall control loop is shown in Figure 1.

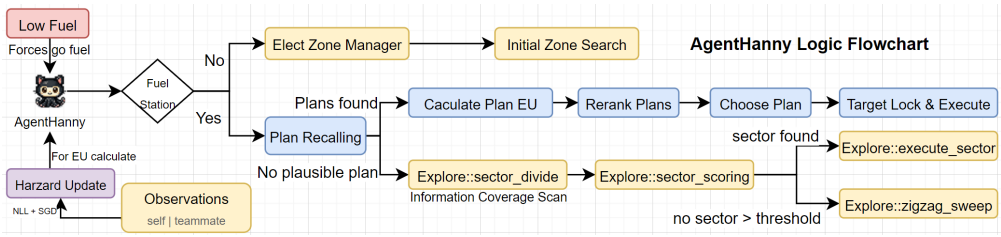


Figure 1: AgentHanny Action Flow

Communications AgentHanny follows G7P2 and integrates incoming team messages. Target locks are renewed while a target remains valid and are released immediately after completion, invalidation, pre-emption, or mode switching (e.g., replanning/refuel).

Memory Layer To support online estimation, the agent keeps augmented memory metadata (e.g., object type and temporal observation cues) synchronized with the shared world model.

Exploration For team-level coverage, the map is partitioned into a dynamic $n \times n$ grid ($n = WorldSize//15$). Sections are scored by teammate density, explored coverage, and travel distance; far infeasible sections are filtered. The best-scoring section is explored first, with zig-zag fallback when no valid section exists.

3.3 AgentEnge - The Priority-Driven Planner

AgentEnge follows a strict four-level priority planner that combines fuel safety, task execution, and structured exploration with team conflict avoidance.

Planning Decisions are evaluated every cycle through a fixed cascade:

1. **Survival** ($fuel < 180$): go to fuel station immediately.
2. **Fill** (carrying tile and valid hole exists): pick the nearest feasible unlocked hole.
3. **Pickup** (capacity available and tile available): lock and collect nearest feasible tile.
4. **Exploration**: sweep the assigned area.

A* planning is accepted only when both fuel margin and object lifetime are feasible. If planning fails, the agent refreshes obstacle knowledge and retries up to $k = 3$ before switching targets.

Communications AgentEnge uses G7P2 target-lock messages to coordinate pickup/fill tasks. Locks are renewed with bounded TTL and inverse-distance priority; higher-priority locks pre-empt local targets to reduce duplicated trips.

Memory & Obstacle Handling Two-position-history horizons suppress oscillation and short cycles; when a cycle is detected, exploration cursor advancement forces escape behavior. Fuel-station observations are explicitly maintained to keep survival planning reliable.

Exploration The agent uses lawnmower sweeps with strip width $2r + 1$, per-agent phase offsets, and zone-aware bootstrapping. After coordination stabilizes, waypoint selection is nudged away from teammate-dense areas to reduce sensing overlap.

3.4 Agent 3 - SmartAgent / The Explorer

SmartAgent is built to maximize team visibility in early and mid stages by prioritizing edge-oriented exploration and broadcasting discovered fuel stations, tiles, and holes.

Core Design Principles Its design follows three principles: safe exploration, fuel-bounded behavior, and collaborative information sharing under the common team protocol.

Think Principles Decision making follows a hierarchy: if no fuel station is known, run serpentine search to bootstrap survival. If $fuel < 150$, refuel immediately; if $150 \leq fuel < 200$, only execute short-range actions that keep the agent near station access. When fuel is safe and no urgent interaction exists, edge exploration is preferred to expand unknown-area coverage quickly.

Exploration strategy The exploration policy biases motion along boundaries and edge structures, which improves frontier discovery versus unconstrained wandering. Navigation uses a staged fallback: constrained A* first, then greedy move selection, and short-depth BFS as the final recovery option.

3.5 AgentSun - The Adaptation Specialist

AgentSun is an adaptation-focused policy designed to remain stable across different map and object configurations without manual retuning.

Planning Its decision cascade is: (1) fuel safety, (2) immediate local action (pick/place on current cell), (3) locked task completion, (4) zone sweep exploration. A lock is kept until target disappearance, unsafe fuel state, or teammate pre-emption.

Adaptive Memory Expiry A two-layer memory policy maintains model freshness: within sensor range, empty observations immediately clear stale entries; beyond range, expiry follows

$$\text{EXPIRY} = \max\big(r,\ 0.7L - \sqrt{A/N}\big),$$

where L is object lifetime, A map area, and N estimated alive objects.

Three-Tier Fuel Management Fuel control combines on-station refuel, critical return protection when margin is tight, and opportunistic refuel when passing near station range (15 cells), with detour-safety checks before tile pursuit.

Exploration Exploration uses boustrophedon sweeps with strip spacing $2r + 1$ for systematic coverage. If blocked, the fallback chain is A* $\rightarrow$ greedy $\rightarrow$ random escape.

3.6 AgentDeng - Utility-Driven Agent

Decision Architecture AgentDeng uses a utility-driven selector rather than a fixed-priority state machine. At each step, four candidates (refuel, hole, tile, explore) are scored and the highest utility is executed:

$$\begin{aligned} U_{\text{refuel}} &= 200 \cdot e^{-4m/60}/(1+d), \\ U_{\text{hole}} &= 50f/(1+d), \\ U_{\text{tile}} &= 30f/(1+d), \\ U_{\text{explore}} &= 8/(1+d), \end{aligned}$$

where $f = t_{\text{left}}/(t_{\text{left}} + d)$ and m is fuel margin. The exponential refuel term balances early over-refuel and late starvation. To avoid oscillation, replanning is triggered only if the target vanishes or a new option exceeds current utility by $1.3\times$.

Memory and Exploration Memory freshness filtering removes candidates unlikely to survive until arrival. A visited-map guides exploration toward nearest unseen regions, and bootstrap behavior uses zone-restricted snake sweeps to find fuel stations quickly.

Anti-Stuck Fallback Path failures are handled by a three-stage fallback: temporary target blacklist (20 steps), greedy one-step recovery, then memory reset and replanning after persistent stagnation.

3.7 AgentWu - Verification and Retry Mechanism

AgentWu centers on verification-driven planning: cache a path, periodically validate target availability, and retry before full replanning.

Planning The planner combines dynamic target selection, conflict-aware locking, and proactive adaptation. After locking a target, it plans and caches a path with constrained A*. If planning fails, the agent performs greedy recovery and retries; after three failed A*+greedy cycles it releases the target, selects a new one, and replans. A target validity check runs every 15 steps and triggers immediate replanning when a target disappears or is claimed by teammates. Without an active target, the agent uses zig-zag exploration and directional fallback to keep moving.

Communication AgentWu follows G7P2, participates in zone coordination, and broadcasts discovered state for team memory sharing.

Fuel management If $fuel < 150$, refuel has immediate priority to avoid deadlock from fuel depletion.

Utility A distance-based utility preference (Manhattan distance) biases selection toward closer feasible targets.

4 Agent Communication & Collaboration

Communication is implemented in `Group7AgentBase` under protocol version `G7P2`, with message tuple $\langle version, step, from, to, type, x, y, payload\rangle$. In `TWEnvironment`, `sense()` and `communicate()` are scheduled before `think()` and `act()`, so messages produced in a step are readable by teammates in that same decision cycle.

The protocol combines sparse events and dense snapshots. Event messages broadcast newly observed objects (`NT/NH/OB`), one-time fuel-station discovery (`FS`), and action confirmations (`PK/FH`). Snapshot message `SS` encodes local empty/occupied cells each step to refresh shared belief quickly. Collaboration is staged: bootstrap zone assignment uses `ZA/ZK` (deterministic manager election with bounded retries), while runtime conflict avoidance uses `TL/TR` target locks with priority and TTL. Together, these mechanisms reduce redundant pursuit and improve task allocation stability under partial observability. And the use of target management enpowers our agents to form a expanding area of exploration and exploitation, illustrating as the figure below.

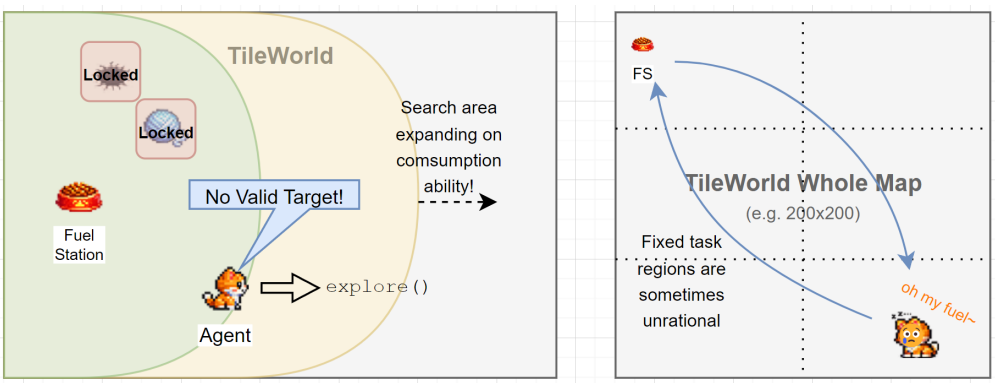


Figure 2: Dynamic Agents Working Area Indication

So that we may minimize step wasted on unnecessary movements $\notin$ {plan execution, exploration}.

5 Result and Analysis

The following table summarizes the mean rewards achieved by our full system across the three parameter sets, averaged over 10 runs each:

	Parameter 1	Parameter 2	Parameter 3
G7 Agent System	1086.5	1397.2	2440.8

Table 2: Mean Rewards Achieved Across Different Environment Settings

Mean reward increases from Parameter Set 1 to 3, suggesting that higher object-generation intensity can outweigh the adverse effects of shorter lifetime and larger map size. The full-system performance serves as the reference point; attribution of gains is analyzed through the ablation results below.

6 Ablation Study

We investigated the contribution on the core designs of our agent system by disabling each of them and test on each configuration for 10 runs and take the mean value for comparison. One is the communication mechanism, we added a `CommunicationBlackhole` that absorbs all outgoing messages to remove communication layer while keeping other parts of the system exact. The other core design is each of our Agent's local policy and planner, we implemented a `MinimalAgent` that only has the basic mobility and survival functions provided by the `Group7AgentBase`, with a simple greedy policy to pick up the nearest tile and fill the nearest hole, without any of the additional logic in the original design.

Then we ran the ablation experiments with the settings and the results are shown in the table below.

Design Ablation	Parameter 1	Parameter 2	Parameter 3
Full System	**1086.5**	**1397.2**	**2440.8**
No Communication	402.2	371.7	61.4
BaseAgent Only	459.5	748.5	545.6

Table 3: Ablation Experiment Results (Mean Rewards over 10 runs)

Ablation reveals the source of improvement. Removing communication reduces reward by 63.0%, 73.4%, and 97.5% for Sets 1–3, respectively, indicating that shared situational awareness is critical in partially observable and highly dynamic settings. Replacing specialized policies with `AgentMinimal` also causes substantial drops (57.7%, 46.4%, 77.6%), showing that common infrastructure alone is insufficient. Overall, team performance depends on both protocol-level coordination and agent-level planning specialization.

7 Conclusion

This project developed a six-agent Tileworld system with a shared engineering backbone (`Group7AgentBase`) and agent-specific decision modules. The framework unifies path planning, fuel-safe control, and structured communication (G7P2), while preserving policy diversity across agents. Empirical results show that the full system consistently outperforms both ablated variants across all tested parameter sets. Ablation further confirms that communication and policy specialization are complementary; removing either causes major reward loss. Limitations remain: richer planners can increase runtime overhead on large maps, and heavy reliance on SS messages raises bandwidth cost. Future work should explore adaptive communication budgets and tighter coupling between global coordination and local plan repair under extreme dynamism.

References

[1] Martha E. Pollack and Marc Ringuette. Introducing the tileworld: Experimentally evaluating agent architectures. In *Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90)*, pages 183–189. AAAI Press, 1990.

[2] Michael Lees. A history of the tileworld agent testbed. Technical Report NOTTCS-WP-2002-1, School of Computer Science and Information Technology, University of Nottingham, March 2002.

[3] David G Kleinbaum and Mitchel Klein. *Survival Analysis: A Self-Learning Text*. Springer Science & Business Media, New York, 3rd edition, 2012.

Appendix

A Codebase

The full codebase is available at `https://github.com/AI6125-Multi-Agent-System-Group-7/tileworld`, with the main branch containing the latesst version of our implementation.

B Presentation Slides

The slides we've used in the presentation session are available at `https://docs.google.com/presentation/d/1kJzGEwVwkM7ECRuiSRMuSTnta10e4aWEWixvR19Tg58`.

C AgentHanny - MLE for Hazard Estimation Derivation

Object survival probability modelled as exponential decay:

$$p_{alive} = e^{-\lambda \Delta t} \quad \in (0, 1]$$

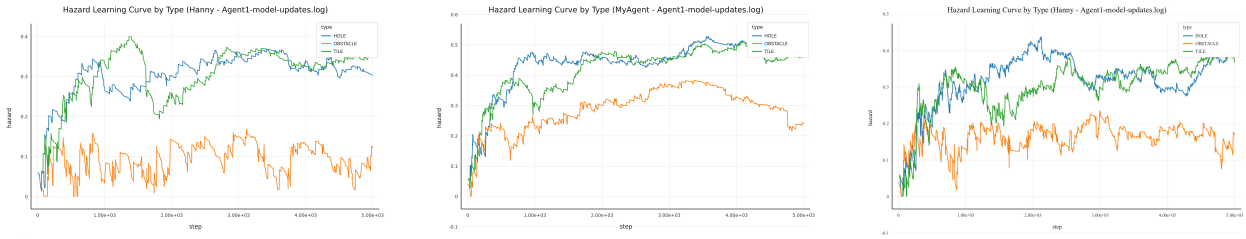
Here, Δt is the estimated time of arrival from the current time stamp, and λ is a reflecting environment $hazard$ that affects the agent's expectation of an object still alive. This $harzard$ will be different across different experiment setups, such as world size and object creation distribution.

We model each observe on an object as a Bernoulli trial with p_{alive}, then we can use the maximum likelihood estimation (MLE) to find the best λ that fits the observation data. By computing the gradient of the negative log-likelihood $l(P)$, we can then perform the MLE via mini-batch stochastic gradient decent (SGD) with update rule as follows:

$$\text{Upon Match} \quad \frac{\partial \ell}{\partial \lambda} = -\Delta t \quad \Longrightarrow \quad \lambda \leftarrow \max\left(0, \lambda - \eta \Delta t\right)$$

$$\text{Upon MisMatch} \quad \frac{\partial \ell}{\partial \lambda} = \frac{\Delta t \, e^{-\lambda \Delta t}}{1 - e^{-\lambda \Delta t}} = \frac{\Delta t}{e^{\lambda \Delta t} - 1} \quad \Longrightarrow \quad \lambda \leftarrow \lambda + \eta \cdot \frac{\Delta t}{e^{\lambda \Delta t} - 1 + \epsilon}$$

By using learning rate schedule and gradient ceiling and clipping, the lambda is then converged to a stable range after 1000 steps on online training, as the curves shown in the figure below.



(a) λ vs Steps - Parameters 1 (b) λ vs Steps - Parameters 2 (c) λ vs Steps - Parameters 3

Figure 3: Hazard Figure Across Different Environment Settings (without smoothing).